Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ

к лабораторной работе №6

на тему:

«ЦИФРОВАЯ ПОДПИСЬ»

БГУИР 6-05-0612-02 74

Выполнил студент группы 353504
ЛЕВШУКОВ Дмитрий Александрович

Проверил ассистент каф. Информатики
ПОЛЯКОВ Кирилл Борисович

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Выполнение работы 4
Вывод 6
Приложение А (обязательное) Исходный код программы 7

ВВЕДЕНИЕ

Электронная цифровая подпись используется для подтверждения подлинности электронных документов и проверки того, что их содержимое не изменялось после подписания.

Цифровая подпись работает с парой ключей. Закрытый ключ хранится только у владельца и используется для формирования подписи. Открытый ключ передаётся другим пользователям и применяется для проверки подписи. Зная открытый ключ, невозможно получить закрытый ключ и сформировать корректную подпись.

В лабораторной работе рассматривается алгоритм формирования и проверки электронной цифровой подписи по ГОСТ 34.10 на эллиптических кривых. Безопасность алгоритма основана на сложности решения задачи дискретного логарифмирования на эллиптической кривой.

Перед формированием подписи для файла вычисляется хеш-значение с помощью алгоритма ГОСТ 34.11-2012 «Стрибог-256». Полученный хеш преобразуется в число e, используемое в дальнейших вычислениях.

При формировании подписи выбирается случайное число k. На эллиптической кривой вычисляется точка $C = kP$, после чего из её координаты получается первая часть подписи r. Вторая часть подписи вычисляется с использованием закрытого ключа d, случайного числа k и хеша e: $s = (r \cdot d + k \cdot e) \bmod q$. Подпись состоит из двух чисел r и s.

При проверке подписи заново вычисляется хеш проверяемого файла. Затем с использованием открытого ключа Q, компонентов подписи и обратного числа к e вычисляются две точки эллиптической кривой. Эти точки складываются. Если координата полученной точки совпадает с r, подпись считается корректной. Если содержимое файла было изменено или подпись повреждена, совпадения не будет.

Задание:

1 Изучить теоретические сведения.

2 Реализовать программное средство формирования и проверки ЭЦП на базе алгоритма ГОСТ 34.10.

1 ВЫПОЛНЕНИЕ РАБОТЫ

Общий алгоритм формирования и проверки электронной цифровой подписи ГОСТ 34.10 представлен в виде блок-схемы на рисунке 1.1.

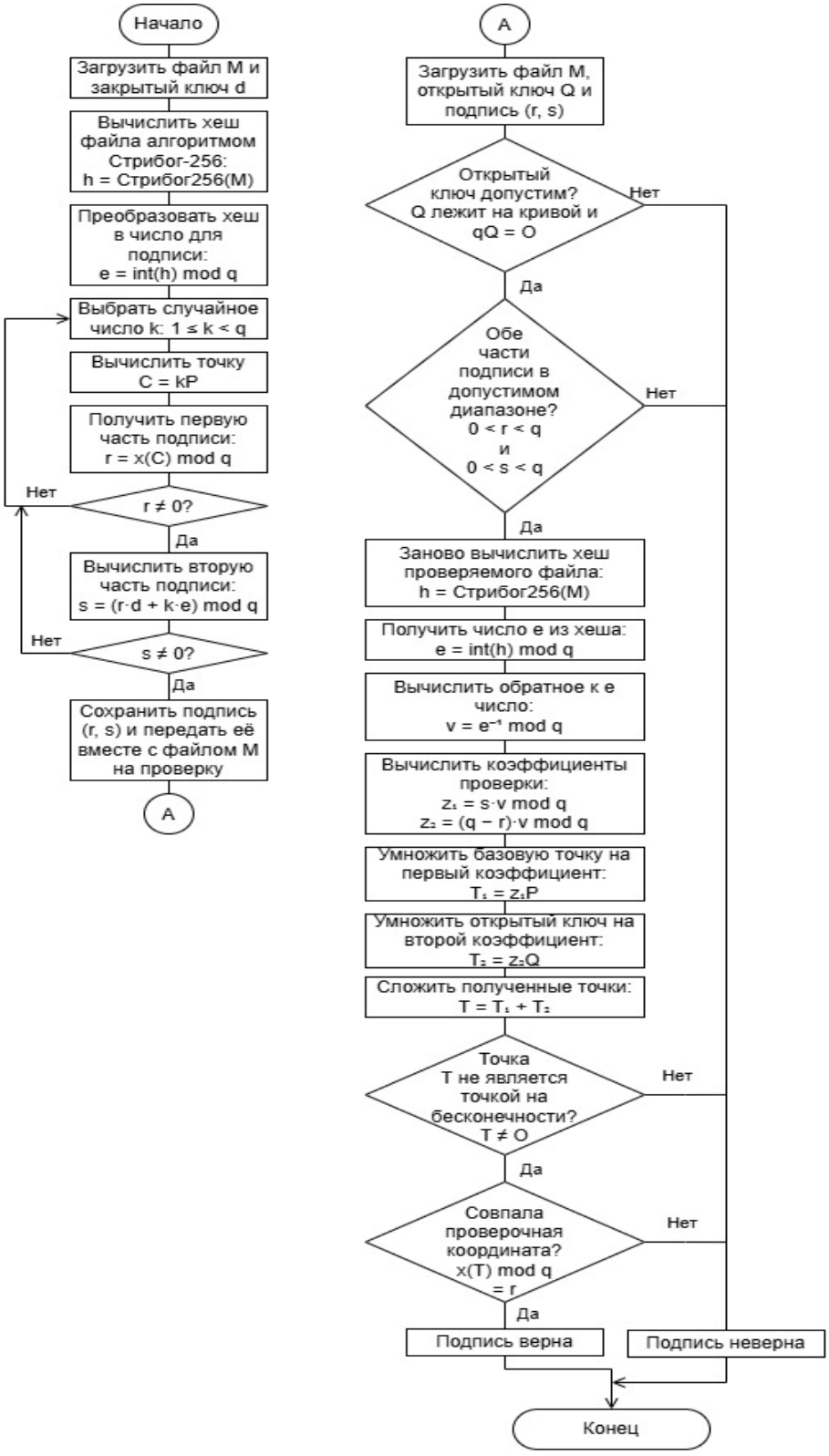


Рисунок 1.1 – Блок-схема алгоритма формирования и проверки электронной цифровой подписи ГОСТ 34.10

Результат выполнения программы в режиме формирования электронной цифровой подписи представлен на рисунке 1.2.

```
PS C:\Prog\BSUIR-Labs\Information Protection Methods\Lab 6> .\Lab6.exe sign private.key test.txt test.sig
Signature created
```

Рисунок 1.2 – Результат выполнения программы в режиме формирования электронной цифровой подписи

Результат успешной проверки электронной цифровой подписи представлен на рисунке 1.3.

```
PS C:\Prog\BSUIR-Labs\Information Protection Methods\Lab 6> .\Lab6.exe verify public.key test.txt test.sig
Signature is valid
```

Рисунок 1.3 – Результат успешной проверки электронной цифровой подписи

Результат неуспешной проверки электронной цифровой подписи представлен на рисунке 1.4.

```
PS C:\Prog\BSUIR-Labs\Information Protection Methods\Lab 6> Copy-Item test.txt tampered.txt
PS C:\Prog\BSUIR-Labs\Information Protection Methods\Lab 6> Add-Content tampered.txt 'Changed content'
PS C:\Prog\BSUIR-Labs\Information Protection Methods\Lab 6> .\Lab6.exe verify public.key tampered.txt test.sig
Signature verification failed
```

Рисунок 1.4 – Результат неуспешной проверки электронной цифровой подписи

ВЫВОД

В ходе выполнения лабораторной работы был изучен алгоритм формирования и проверки электронной цифровой подписи по ГОСТ 34.10 на эллиптических кривых.

В программном средстве реализованы генерация пары ключей, формирование подписи файла и проверка её подлинности. Для получения хеш-значения сообщения использован алгоритм ГОСТ 34.11-2012 (Стрибог-256), а операции с эллиптическими кривыми и вычисления компонентов подписи реализованы на языке C++.

В процессе тестирования была создана корректная цифровая подпись файла, после чего она успешно прошла проверку. При изменении содержимого файла проверка завершилась отрицательно, что подтверждает обнаружение изменений и работоспособность алгоритма.

Таким образом, были закреплены навыки работы с хеш-функциями, открытыми и закрытыми ключами, эллиптическими кривыми, формирования и проверки электронной цифровой подписи.

ПРИЛОЖЕНИЕ А
(обязательное)
Исходный код программы

```
#include "crypto.hpp"

#include <iostream>

namespace {

void printUsage(const char* program)
{
    std::cerr << "Usage:\n"
              << "    " << program << " keygen <private_key_file> <public_key_file>\n"
              << "  " << program << " sign <private_key_file> <input_file> <signature_file>\n"
              << "    " << program << " verify <public_key_file> <input_file> <signature_file>\n";
}

}

int main(int argc, char* argv[])
{
    if (argc == 4 && std::string(argv[1]) == "keygen") {
        if (!gost::generateKeys(argv[2], argv[3])) {
            std::cerr << "Error: key generation failed\n";
            return 1;
        }
        std::cout << "Key pair generated\n";
        return 0;
    }

    if (argc == 5 && std::string(argv[1]) == "sign") {
        if (!gost::signFile(argv[2], argv[3], argv[4])) {
            std::cerr << "Error: signature generation failed\n";
            return 1;
        }
        std::cout << "Signature created\n";
        return 0;
    }

    if (argc == 5 && std::string(argv[1]) == "verify") {
        if (!gost::verifyFile(argv[2], argv[3], argv[4])) {
            std::cerr << "Signature verification failed\n";
            return 2;
        }
        std::cout << "Signature is valid\n";
        return 0;
    }

    printUsage(argv[0]);
    return 1;
}

#pragma once

#define NOMINMAX
#include <windows.h>
#include <bcrypt.h>
```

```
#include <algorithm>
#include <array>
#include <cstdint>
#include <cstring>
#include <fstream>
#include <iomanip>
#include <sstream>
#include <string>
#include <vector>

namespace gost {

using Byte = std::uint8_t;
using Bytes = std::vector<Byte>;

struct UInt256 {
    std::array<std::uint64_t, 4> limb{};

    bool zero() const
    {
        return limb[0] == 0 && limb[1] == 0 && limb[2] == 0 && limb[3] ==
0;
    }

    bool bit(unsigned index) const
    {
        return ((limb[index / 64] >> (index % 64)) & 1U) != 0;
    }
};

inline int compare(const UInt256& left, const UInt256& right)
{
    for (int i = 3; i >= 0; --i) {
        if (left.limb[i] < right.limb[i]) return -1;
        if (left.limb[i] > right.limb[i]) return 1;
    }
    return 0;
}

inline UInt256 addRaw(const UInt256& left, const UInt256& right, bool&
carry)
{
    UInt256 result;
    carry = false;
    for (unsigned i = 0; i < 4; ++i) {
        const std::uint64_t first = left.limb[i] + right.limb[i];
        const bool firstCarry = first < left.limb[i];
                    const    std::uint64_t    second    =    first    +
static_cast<std::uint64_t>(carry);
        const bool secondCarry = second < first;
        result.limb[i] = second;
        carry = firstCarry || secondCarry;
    }
    return result;
}

inline UInt256 subRaw(const UInt256& left, const UInt256& right)
{
    UInt256 result;
    bool borrow = false;
    for (unsigned i = 0; i < 4; ++i) {
        const std::uint64_t first = left.limb[i] - right.limb[i];
        const bool firstBorrow = left.limb[i] < right.limb[i];
```

```
                            const    std::uint64_t    second    =    first    -
static_cast<std::uint64_t>(borrow);
                             const    bool    secondBorrow    =    first    <
static_cast<std::uint64_t>(borrow);
                result.limb[i] = second;
                borrow = firstBorrow || secondBorrow;
            }
            return result;
        }

        inline int hexDigit(char value)
        {
            if (value >= '0' && value <= '9') return value - '0';
            if (value >= 'a' && value <= 'f') return value - 'a' + 10;
            if (value >= 'A' && value <= 'F') return value - 'A' + 10;
            return -1;
        }

        inline bool fromHex(const std::string& text, UInt256& value)
        {
            if (text.size() != 64) return false;
            value = {};
            for (std::size_t i = 0; i < text.size(); ++i) {
                const int nibble = hexDigit(text[text.size() - 1 - i]);
                if (nibble < 0) return false;
                 value.limb[i / 16] |= static_cast<std::uint64_t>(nibble) << ((i
% 16) * 4);
            }
            return true;
        }

        inline UInt256 fromBytes(const std::array<Byte, 32>& bytes)
        {
            UInt256 value;
            for (unsigned i = 0; i < bytes.size(); ++i) {
                value.limb[(31 - i) / 8] |=
                    static_cast<std::uint64_t>(bytes[i]) << (((31 - i) % 8) * 8);
            }
            return value;
        }

        inline std::string toHex(const UInt256& value)
        {
            static constexpr char digits[] = "0123456789abcdef";
            std::string result(64, '0');
            for (unsigned i = 0; i < 32; ++i) {
                const Byte byte = static_cast<Byte>(value.limb[(31 - i) / 8] >>
(((31 - i) % 8) * 8));
                result[i * 2] = digits[byte >> 4];
                result[i * 2 + 1] = digits[byte & 0x0f];
            }
            return result;
        }

        inline UInt256 addMod(const UInt256& left, const UInt256& right, const
UInt256& modulus)
        {
            bool carry = false;
            UInt256 sum = addRaw(left, right, carry);
            if (carry || compare(sum, modulus) >= 0) sum = subRaw(sum, modulus);
            return sum;
        }
```

```
      inline UInt256 subMod(const UInt256& left, const UInt256& right, const
UInt256& modulus)
      {
          if (compare(left, right) >= 0) return subRaw(left, right);
          return subRaw(modulus, subRaw(right, left));
      }

      inline UInt256 reduceMod(const UInt256& value, const UInt256& modulus)
      {
          UInt256 reduced;
          UInt256 one;
          one.limb[0] = 1;
          for (int bit = 255; bit >= 0; --bit) {
              reduced = addMod(reduced, reduced, modulus);
              if (value.bit(static_cast<unsigned>(bit))) {
                  reduced = addMod(reduced, one, modulus);
              }
          }
          return reduced;
      }

      inline UInt256 mulMod(const UInt256& left, const UInt256& right, const
UInt256& modulus)
      {
          UInt256 result;
          UInt256 addend = left;
          for (unsigned bit = 0; bit < 256; ++bit) {
              if (right.bit(bit)) result = addMod(result, addend, modulus);
              if (bit != 255) addend = addMod(addend, addend, modulus);
          }
          return result;
      }

      inline UInt256 powMod(UInt256 base, const UInt256& exponent, const
UInt256& modulus)
      {
          UInt256 result;
          result.limb[0] = 1;
          for (unsigned bit = 0; bit < 256; ++bit) {
              if (exponent.bit(bit)) result = mulMod(result, base, modulus);
              if (bit != 255) base = mulMod(base, base, modulus);
          }
          return result;
      }

      inline UInt256 subtractSmall(UInt256 value, std::uint64_t amount)
      {
          UInt256 small;
          small.limb[0] = amount;
          return subRaw(value, small);
      }

      inline const UInt256& field()
      {
          static const UInt256 value = [] {
              UInt256 result;
              fromHex("fffffffffffffffffffffffffffffffffffffffffffffffffffffffe
ffffffd97", result);
              return result;
          }();
          return value;
      }

      inline const UInt256& order()
```

```
    {
        static const UInt256 value = [] {
            UInt256 result;
            fromHex("400000000000000000000000000000000fd8cddfc87b6635c115af5
56c360c67", result);
            return result;
        }();
        return value;
    }

    inline const UInt256& curveA()
    {
        static const UInt256 value = [] {
            UInt256 result;
            fromHex("c2173f1513981673af4892c23035a27ce25e2013bf95aa33b22c656
f277e7335", result);
            return result;
        }();
        return value;
    }

    inline const UInt256& generatorX()
    {
        static const UInt256 value = [] {
            UInt256 result;
            fromHex("91e38443a5e82c0d880923425712b2bb658b9196932e02c78b2582f
e742daa28", result);
            return result;
        }();
        return value;
    }

    inline const UInt256& generatorY()
    {
        static const UInt256 value = [] {
            UInt256 result;
            fromHex("32879423ab1a0375895786c4bb46e9565fde0b5344766740af268ad
b32322e5c", result);
            return result;
        }();
        return value;
    }

    struct Point {
        UInt256 x;
        UInt256 y;
        UInt256 z;
        bool infinity = true;
    };

    inline Point pointAtInfinity()
    {
        return {};
    }

    inline Point affinePoint(const UInt256& x, const UInt256& y)
    {
        Point result;
        result.x = x;
        result.y = y;
        result.z.limb[0] = 1;
        result.infinity = false;
        return result;
    }
```

```
    inline Point pointDouble(const Point& point)
    {
        if (point.infinity || point.y.zero()) return pointAtInfinity();

        const UInt256& p = field();
        const UInt256 xx = mulMod(point.x, point.x, p);
        const UInt256 yy = mulMod(point.y, point.y, p);
        const UInt256 yyyy = mulMod(yy, yy, p);
        const UInt256 zz = mulMod(point.z, point.z, p);
        const UInt256 s = mulMod(
            UInt256{{4, 0, 0, 0}},
            mulMod(point.x, yy, p), p);
        const UInt256 m = addMod(
            mulMod(UInt256{{3, 0, 0, 0}}, xx, p),
            mulMod(curveA(), mulMod(zz, zz, p), p), p);
        const UInt256 x3 = subMod(mulMod(m, m, p), addMod(s, s, p), p);
        const UInt256 eightYyyy = mulMod(UInt256{{8, 0, 0, 0}}, yyyy, p);
        const UInt256 y3 = subMod(
            mulMod(m, subMod(s, x3, p), p), eightYyyy, p);
        const UInt256 yPlusZ = addMod(point.y, point.z, p);
        const UInt256 z3 = subMod(
            subMod(mulMod(yPlusZ, yPlusZ, p), yy, p), zz, p);

        return Point{x3, y3, z3, false};
    }

    inline Point pointAdd(const Point& left, const Point& right)
    {
        if (left.infinity) return right;
        if (right.infinity) return left;
        const UInt256& p = field();

        const UInt256 z1z1 = mulMod(left.z, left.z, p);
        const UInt256 z2z2 = mulMod(right.z, right.z, p);
        const UInt256 u1 = mulMod(left.x, z2z2, p);
        const UInt256 u2 = mulMod(right.x, z1z1, p);
        const UInt256 s1 = mulMod(left.y, mulMod(right.z, z2z2, p), p);
        const UInt256 s2 = mulMod(right.y, mulMod(left.z, z1z1, p), p);

        if (compare(u1, u2) == 0) {
                return compare(s1, s2) == 0 ? pointDouble(left) :
pointAtInfinity();
        }

        const UInt256 h = subMod(u2, u1, p);
        const UInt256 i = mulMod(UInt256{{4, 0, 0, 0}}, mulMod(h, h, p), p);
        const UInt256 j = mulMod(h, i, p);
         const UInt256 r = mulMod(UInt256{{2, 0, 0, 0}}, subMod(s2, s1, p),
p);
        const UInt256 v = mulMod(u1, i, p);
        const UInt256 x3 = subMod(subMod(mulMod(r, r, p), j, p), addMod(v, v,
p), p);
        const UInt256 y3 = subMod(
            mulMod(r, subMod(v, x3, p), p),
            mulMod(UInt256{{2, 0, 0, 0}}, mulMod(s1, j, p), p), p);
        const UInt256 zSum = addMod(left.z, right.z, p);
        const UInt256 z3 = mulMod(
            subMod(subMod(mulMod(zSum, zSum, p), z1z1, p), z2z2, p), h, p);
        return Point{x3, y3, z3, false};
    }

    inline Point scalarMultiply(const Point& point, const UInt256& scalar)
    {
```

```
        Point result = pointAtInfinity();
        Point addend = point;
        for (unsigned bit = 0; bit < 256; ++bit) {
            if (scalar.bit(bit)) result = pointAdd(result, addend);
            if (bit != 255) addend = pointDouble(addend);
        }
        return result;
    }

    inline bool toAffine(const Point& point, UInt256& x, UInt256& y)
    {
        if (point.infinity) return false;
        const UInt256 zInverse = powMod(point.z, subtractSmall(field(), 2),
field());
        const UInt256 zInverse2 = mulMod(zInverse, zInverse, field());
        const UInt256 zInverse3 = mulMod(zInverse2, zInverse, field());
        x = mulMod(point.x, zInverse2, field());
        y = mulMod(point.y, zInverse3, field());
        return true;
    }

    inline bool isOnCurve(const UInt256& x, const UInt256& y)
    {
         if (compare(x, field()) >= 0 || compare(y, field()) >= 0) return
false;
        const UInt256 left = mulMod(y, y, field());
        const UInt256 x2 = mulMod(x, x, field());
        const UInt256 x3 = mulMod(x2, x, field());
        UInt256 b;
        fromHex("295f9bae7428ed9ccc20e7c359a9d41a22fccd9108e17bf7ba9337a6f8a
e9513", b);
        const UInt256 right = addMod(
            addMod(x3, mulMod(curveA(), x, field()), field()),
            b, field());
        return compare(left, right) == 0;
    }

    inline bool randomScalar(UInt256& result)
    {
        std::array<Byte, 32> bytes{};
        do {
                            if     (BCryptGenRandom(nullptr,     bytes.data(),
static_cast<ULONG>(bytes.size()),
                                    BCRYPT_USE_SYSTEM_PREFERRED_RNG) != 0) {
                return false;
            }
            result = fromBytes(bytes);
        } while (result.zero() || compare(result, order()) >= 0);
        return true;
    }

    class Streebog {
        private:
        inline static constexpr std::array<Byte, 256> kPi = {
            252,238,221,17,207,110,49,22,251,196,250,218,35,197,4,77,
            233,119,240,219,147,46,153,186,23,54,241,187,20,205,95,193,
            249,24,101,90,226,92,239,33,129,28,60,66,139,1,142,79,
            5,132,2,174,227,106,143,160,6,11,237,152,127,212,211,31,
            235,52,44,81,234,200,72,171,242,42,104,162,253,58,206,204,
            181,112,14,86,8,12,118,18,191,114,19,71,156,183,93,135,
            21,161,150,41,16,123,154,199,243,145,120,111,157,158,178,177,
            50,117,25,61,255,53,138,126,109,84,198,128,195,189,13,87,
            223,245,36,169,62,168,67,201,215,121,214,246,124,34,185,3,
            224,15,236,222,122,148,176,188,220,232,40,80,78,51,10,74,
```

```
                167,151,96,115,30,0,98,68,26,184,56,130,100,159,38,65,
                173,69,70,146,39,94,85,47,140,163,165,125,105,213,149,59,
                7,88,179,64,134,172,29,247,48,55,107,228,136,217,231,137,
                225,27,131,73,76,63,248,254,141,83,170,144,202,216,133,97,
                32,113,103,164,45,43,9,91,203,155,37,208,190,229,108,82,
                89,166,116,210,230,244,180,192,209,102,175,194,57,75,99,182
            };

            inline static constexpr std::array<std::uint64_t, 64> kA = {
                0x8e20faa72ba0b470ULL,0x47107ddd9b505a38ULL,0xad08b0e0c3282d1cUL
L,0xd8045870ef14980eULL,
                0x6c022c38f90a4c07ULL,0x3601161cf205268dULL,0x1b8e0b0e798c13c8UL
L,0x83478b07b2468764ULL,
                0xa011d380818e8f40ULL,0x5086e740ce47c920ULL,0x2843fd2067adea10UL
L,0x14aff010bdd87508ULL,
                0x0ad97808d06cb404ULL,0x05e23c0468365a02ULL,0x8c711e02341b2d01UL
L,0x46b60f011a83988eULL,
                0x90dab52a387ae76fULL,0x486dd4151c3dfdb9ULL,0x24b86a840e90f0d2UL
L,0x125c354207487869ULL,
                0x092e94218d243cbaULL,0x8a174a9ec8121e5dULL,0x4585254f64090fa0UL
L,0xaccc9ca9328a8950ULL,
                0x9d4df05d5f661451ULL,0xc0a878a0a1330aa6ULL,0x60543c50de970553UL
L,0x302a1e286fc58ca7ULL,
                0x18150f14b9ec46ddULL,0x0c84890ad27623e0ULL,0x0642ca05693b9f70UL
L,0x0321658cba93c138ULL,
                0x86275df09ce8aaa8ULL,0x439da0784e745554ULL,0xafc0503c273aa42aUL
L,0xd960281e9d1d5215ULL,
                0xe230140fc0802984ULL,0x71180a8960409a42ULL,0xb60c05ca30204d21UL
L,0x5b068c651810a89eULL,
                0x456c34887a3805b9ULL,0xac361a443d1c8cd2ULL,0x561b0d22900e4669UL
L,0x2b838811480723baULL,
                0x9bcf4486248d9f5dULL,0xc3e9224312c8c1a0ULL,0xeffa11af0964ee50UL
L,0xf97d86d98a327728ULL,
                0xe4fa2054a80b329cULL,0x727d102a548b194eULL,0x39b008152acb8227UL
L,0x9258048415eb419dULL,
                0x492c024284fbaec0ULL,0xaa16012142f35760ULL,0x550b8e9e21f7a530UL
L,0xa48b474f9ef5dc18ULL,
                0x70a6a56e2440598eULL,0x3853dc371220a247ULL,0x1ca76e95091051adUL
L,0x0edd37c48a08a6d8ULL,
                0x07e095624504536cULL,0x8d70c431ac02a736ULL,0xc83862965601dd1bUL
L,0x641c314b2b8ee083ULL
            };

            static const std::array<std::array<Byte, 64>, 12>& roundConstants()
            {
                static const std::array<std::array<Byte, 64>, 12> values = [] {
                    constexpr const char* hex[12] = {
                        "b1085bda1ecadae9ebcb2f81c0657c1f2f6a76432e45d016714eb88
d7585c4fc4b7ce09192676901a2422a08a460d31505767436cc744d23dd806559f2a64507",
                        "6fa3b58aa99d2f1a4fe39d460f70b5d7f3feea720a232b9861d55e0
f16b501319ab5176b12d699585cb561c2db0aa7ca55dda21bd7cbcd56e679047021b19bb7",
                        "f574dcac2bce2fc70a39fc286a3d843506f15e5f529c1f8bf2ea751
4b1297b7bd3e20fe490359eb1c1c93a376062db09c2b6f443867adb31991e96f50aba0ab2",
                        "ef1fdfb3e81566d2f948e1a05d71e4dd488e857e335c3c7d9d721ca
d685e353fa9d72c82ed03d675d8b71333935203be3453eaa193e837f1220cbebc84e3d12e",
                        "4bea6bacad4747999a3f410c6ca923637f151c1f1686104a359e35d
7800fffbdbfcd1747253af5a3dfff00b723271a167a56a27ea9ea63f5601758fd7c6cfe57",
                        "ae4faeae1d3ad3d96fa4c33b7a3039c02d66c4f95142a46c187f9ab
49af08ec6cffaa6b71c9ab7b40af21f66c2bec6b6bf71c57236904f35fa68407a46647d6e",
                        "f4c70e16eeaac5ec51ac86febf240954399ec6c7e6bf87c9d3473e3
3197a93c90992abc52d822c3706476983284a05043517454ca23c4af38886564d3a14d493",
                        "9b1f5b424d93c9a703e7aa020c6e41414eb7f8719c36de1e89b4443
b4ddbc49af4892bcb929b069069d18d2bd1a5c42f36acc2355951a8d9a47f0dd4bf02e71e",
```

```
                    "378f5a541631229b944c9ad8ec165fde3a7d3a1b258942243cd955b
7e00d0984800a440bdbb2ceb17b2b8a9aa6079c540e38dc92cb1f2a607261445183235adb",
                    "abbedea680056f52382ae548b2e4f3f38941e71cff8a78db1fffe18
a1b3361039fe76702af69334b7a1e6c303b7652f43698fad1153bb6c374b4c7fb98459ced",
                    "7bcd9ed0efc889fb3002c6cd635afe94d8fa6bbbebab07612001802
1148466798a1d71efea48b9caefbacd1d7d476e98dea2594ac06fd85d6bcaa4cd81f32d1b",
                    "378ee767f11631bad21380b00449b17acda43c32bcdf1d77f82012d
430219f9b5d80ef9d1891cc86e71da4aa88e12852faf417d5d9b21b9948bc924af11bd720"
                };
                std::array<std::array<Byte, 64>, 12> result{};
                for (unsigned row = 0; row < result.size(); ++row) {
                    for (unsigned i = 0; i < 64; ++i) {
                        result[row][i] = static_cast<Byte>(
                                    (hexDigit(hex[row][2 * i]) << 4) |
hexDigit(hex[row][2 * i + 1]));
                    }
                }
                return result;
            }();
            return values;
        }

        using StreebogBlock = std::array<Byte, 64>;

        static void xorInto(StreebogBlock& target, const StreebogBlock& value)
        {
             for (std::size_t i = 0; i < target.size(); ++i) target[i] ^=
value[i];
        }

        static void lps(StreebogBlock& state)
        {
            static constexpr std::array<Byte, 64> tau = {
                0,8,16,24,32,40,48,56,1,9,17,25,33,41,49,57,
                2,10,18,26,34,42,50,58,3,11,19,27,35,43,51,59,
                4,12,20,28,36,44,52,60,5,13,21,29,37,45,53,61,
                6,14,22,30,38,46,54,62,7,15,23,31,39,47,55,63
            };
            StreebogBlock substituted{};
             for (std::size_t i = 0; i < state.size(); ++i) substituted[i] =
kPi[state[i]];

            StreebogBlock permuted{};
             for (std::size_t i = 0; i < state.size(); ++i) permuted[i] =
substituted[tau[i]];

            StreebogBlock transformed{};
            for (std::size_t group = 0; group < 8; ++group) {
                std::uint64_t word = 0;
                for (std::size_t byte = 0; byte < 8; ++byte) {
                    const Byte value = permuted[group * 8 + byte];
                    for (std::size_t bit = 0; bit < 8; ++bit) {
                        if ((value & (0x80U >> bit)) != 0) word ^= kA[byte *
8 + bit];
                    }
                }
                for (std::size_t byte = 0; byte < 8; ++byte) {
                    transformed[group * 8 + byte] =
                        static_cast<Byte>(word >> (56 - 8 * byte));
                }
            }
            state = transformed;
        }
```

```
        static StreebogBlock gostEncrypt(const StreebogBlock& key,
                                         const StreebogBlock& message)
        {
            StreebogBlock state = message;
            xorInto(state, key);

            StreebogBlock roundKey = key;
             for (std::size_t round = 0; round < roundConstants().size();
++round) {
                lps(state);
                xorInto(roundKey, roundConstants()[round]);
                lps(roundKey);
                xorInto(state, roundKey);
            }
            return state;
        }

        static StreebogBlock addBlocks(const StreebogBlock& left,
                                       const StreebogBlock& right)
        {
            StreebogBlock result{};
            unsigned carry = 0;
            for (std::size_t i = result.size(); i-- > 0;) {
               const unsigned sum = static_cast<unsigned>(left[i]) + right[i]
+ carry;
                result[i] = static_cast<Byte>(sum);
                carry = sum >> 8;
            }
            return result;
        }

        static void addBits(StreebogBlock& value, std::uint64_t addend)
        {
            for (std::size_t i = value.size(); i-- > 0 && addend != 0;) {
                const unsigned sum = static_cast<unsigned>(value[i]) +
                                     static_cast<unsigned>(addend & 0xffU);
                value[i] = static_cast<Byte>(sum);
                addend = (addend >> 8) + (sum >> 8);
            }
        }

        static StreebogBlock compress(const StreebogBlock& hash,
                                      const StreebogBlock& counter,
                                      const StreebogBlock& message)
        {
            StreebogBlock key = hash;
            xorInto(key, counter);
            lps(key);

            StreebogBlock result = gostEncrypt(key, message);
            xorInto(result, hash);
            xorInto(result, message);
            return result;
        }

        public:
        static std::array<Byte, 32> hash(const Bytes& message)
        {
            StreebogBlock hash{};
            hash.fill(1);
            StreebogBlock counter{};
            StreebogBlock sigma{};
            std::size_t offset = 0;
```

```
            while (message.size() - offset >= 64) {
                StreebogBlock block{};
                                        std::reverse_copy(message.begin()     +
static_cast<std::ptrdiff_t>(offset),
                                                                message.begin()   +
static_cast<std::ptrdiff_t>(offset + 64),
                                     block.begin());
                hash = compress(hash, counter, block);
                addBits(counter, 512);
                sigma = addBlocks(sigma, block);
                offset += 64;
            }

            const std::size_t remainder = message.size() - offset;
            StreebogBlock finalBlock{};
            finalBlock[63 - remainder] = 1;
                                      std::reverse_copy(message.begin()       +
static_cast<std::ptrdiff_t>(offset),
                                message.end(),
                                                          finalBlock.begin()   +
static_cast<std::ptrdiff_t>(64 - remainder));

            hash = compress(hash, counter, finalBlock);
            addBits(counter, static_cast<std::uint64_t>(remainder) * 8);
            sigma = addBlocks(sigma, finalBlock);

            const StreebogBlock zero{};
            hash = compress(hash, zero, counter);
            hash = compress(hash, zero, sigma);

            std::array<Byte, 32> digest{};
            for (std::size_t i = 0; i < digest.size(); ++i) {
                digest[i] = hash[digest.size() - 1 - i];
            }
            return digest;
        }

        static bool hashFile(const std::string& path, UInt256& hashValue)
        {
            std::ifstream input(path, std::ios::binary);
            if (!input) return false;
                   Bytes   contents((std::istreambuf_iterator<char>(input)),
std::istreambuf_iterator<char>());
            if (!input.eof() && input.fail()) return false;
            const auto digest = hash(contents);
            hashValue = fromBytes(digest);
            return true;
        }

        static bool hashToScalar(const std::string& path, UInt256& result)
        {
            if (!hashFile(path, result)) return false;
            result = reduceMod(result, order());
            if (result.zero()) result.limb[0] = 1;
            return true;
        }

    };

    class Gost3410Signature {
        private:
        static bool parseFields(const std::string& path,
                                const std::vector<std::string>& names,
                                std::vector<UInt256>& values)
```

```
        {
            std::ifstream input(path);
            if (!input) return false;
            values.clear();
            std::string line;
            for (const auto& name : names) {
                if (!std::getline(input, line)) return false;
                if (!line.empty() && line.back() == '\r') line.pop_back();
                if (line.compare(0, name.size() + 1, name + "=") != 0) return
false;
                UInt256 value;
                  if (!fromHex(line.substr(name.size() + 1), value)) return
false;
                values.push_back(value);
            }
            return !std::getline(input, line);
        }

        static bool writeFields(const std::string& path,
                                const std::vector<std::pair<std::string,
UInt256>>& fields)
        {
            std::ofstream output(path, std::ios::trunc);
            if (!output) return false;
            for (const auto& field : fields) output << field.first << '=' <<
toHex(field.second) << '\n';
            return output.good();
        }

        public:
          static bool generateKeys(const std::string& privatePath, const
std::string& publicPath)
        {
            if (privatePath == publicPath) return false;
            UInt256 privateKey;
            UInt256 publicX, publicY;
            if (!randomScalar(privateKey) ||
                        !toAffine(scalarMultiply(affinePoint(generatorX(),
generatorY()), privateKey),
                        publicX, publicY)) return false;
            return writeFields(privatePath, {{"d", privateKey}}) &&
                   writeFields(publicPath, {{"x", publicX}, {"y", publicY}});
        }

        static bool signFile(const std::string& privatePath,
                             const std::string& inputPath,
                             const std::string& signaturePath)
        {
             if (signaturePath == privatePath || signaturePath == inputPath)
return false;
            std::vector<UInt256> privateFields;
            UInt256 hash;
            if (!parseFields(privatePath, {"d"}, privateFields) ||
               privateFields[0].zero() || compare(privateFields[0], order())
>= 0 ||
                !Streebog::hashToScalar(inputPath, hash)) return false;

            for (;;) {
                UInt256 nonce;
                if (!randomScalar(nonce)) return false;
                UInt256 pointX, pointY;
                     if  (!toAffine(scalarMultiply(affinePoint(generatorX(),
generatorY()), nonce),
                              pointX, pointY)) return false;
```

```
                const UInt256 r = reduceMod(pointX, order());
                if (r.zero()) continue;
                const UInt256 rd = mulMod(r, privateFields[0], order());
                const UInt256 ke = mulMod(nonce, hash, order());
                const UInt256 s = addMod(rd, ke, order());
                if (s.zero()) continue;
                return writeFields(signaturePath, {{"r", r}, {"s", s}});
            }
        }
        static bool verifyFile(const std::string& publicPath,
                               const std::string& inputPath,
                               const std::string& signaturePath)
        {
            std::vector<UInt256> publicFields, signatureFields;
            UInt256 hash;
            if (!parseFields(publicPath, {"x", "y"}, publicFields) ||
                !parseFields(signaturePath, {"r", "s"}, signatureFields) ||
                !isOnCurve(publicFields[0], publicFields[1]) ||
                                !scalarMultiply(affinePoint(publicFields[0],
publicFields[1]), order()).infinity ||
                   signatureFields[0].zero() || compare(signatureFields[0],
order()) >= 0 ||
                   signatureFields[1].zero() || compare(signatureFields[1],
order()) >= 0 ||
                !Streebog::hashToScalar(inputPath, hash)) return false;

             const UInt256 inverse = powMod(hash, subtractSmall(order(), 2),
order());
            const UInt256 z1 = mulMod(signatureFields[1], inverse, order());
             const UInt256 z2 = mulMod(subRaw(order(), signatureFields[0]),
inverse, order());
                  const  Point  publicKey  =  affinePoint(publicFields[0],
publicFields[1]);
            const Point point = pointAdd(
                scalarMultiply(affinePoint(generatorX(), generatorY()), z1),
                scalarMultiply(publicKey, z2));
            UInt256 pointX, pointY;
            if (!toAffine(point, pointX, pointY)) return false;
            pointX = reduceMod(pointX, order());
            return compare(pointX, signatureFields[0]) == 0;
        }

    };
    inline   bool   generateKeys(const   std::string&   privatePath,   const
std::string& publicPath)
    {
        return Gost3410Signature::generateKeys(privatePath, publicPath);
    }
    inline bool signFile(const std::string& privatePath,
                         const std::string& inputPath,
                         const std::string& signaturePath)
    {
             return   Gost3410Signature::signFile(privatePath,   inputPath,
signaturePath);
    }
    inline bool verifyFile(const std::string& publicPath,
                           const std::string& inputPath,
                           const std::string& signaturePath)
    {
            return   Gost3410Signature::verifyFile(publicPath,   inputPath,
signaturePath);
    }

    }
```